

FinGuard AI: Real-Time Financial Fraud Detection System

Software Engineering Project Proposal

Submitted by: Arnav Kathuria (1024030068) and Akshit Shandilya (1024030067)

Department of Computer Engineering

Thapar Institute of Engineering and Technology

August 18, 2026

1 Higher-order Goal

This project aims to strengthen the integrity and resilience of digital financial systems by making fraud detection faster, more accurate, and fully explainable. Beyond the immediate engineering objective, it addresses a broader concern in financial technology: as transaction volumes and fraud sophistication grow, institutions need adaptive systems that regulators, analysts, and customers can trust and audit.

2 Introduction / Problem Statement

Financial institutions rely heavily on transaction-monitoring systems to catch fraud before funds are lost. Most legacy systems, however, are built around static, hand-written rule engines that compare each transaction against a fixed rulebook.

This approach breaks down in predictable ways. Static rules cannot detect fraud patterns nobody has explicitly coded for, so novel schemes slip through until an analyst manually writes a new rule. Blunt thresholds treat every signal with equal weight, generating high false-positive rates that block legitimate customers as often as they catch fraudsters. Batch-style checks introduce detection lag, so fraud is frequently caught only after funds have already moved. Every flagged case still requires an analyst to manually piece together evidence, and the systems offer little to no explanation for why a transaction was blocked, which is increasingly a regulatory liability.

FinGuard AI addresses this gap by combining deterministic rules with supervised and unsupervised machine learning and behavioral analysis in a single explainable, real-time scoring pipeline.

3 Objectives

- Build a real-time risk-scoring engine that evaluates each transaction as it occurs.
- Combine deterministic rules with supervised (XGBoost) and unsupervised (Isolation Forest) models for hybrid fraud detection.
- Generate explainable decisions using per-transaction feature attribution (SHAP) rather than opaque scores.
- Automate evidence assembly for analyst investigation instead of fully manual case-building.
- Support continuous model improvement through an analyst-feedback loop.

4 Methodology / Solution Approach

4.1 System Architecture and Pipeline

FinGuard AI layers deterministic rules, a supervised fraud model, unsupervised anomaly detection, behavioral analysis, and explainability into one continuous pipeline that carries each transaction from ingestion to decision:

- Customer Frontend → API Gateway (JWT authentication + RBAC) → Transaction Service → Feature Extraction.
- Parallel detection stage: a deterministic Rule Engine runs alongside a supervised XGBoost fraud model and an unsupervised Isolation Forest anomaly detector.
- Risk Aggregation combines the three signals into a single Final Risk Score (0–100).
- The scored transaction and its evidence are surfaced on an Analyst Dashboard for review.
- Analyst decisions (confirmed fraud / false positive) are written back to a Decision Feedback Database, closing the loop for continuous retraining.

4.2 Key Model Features

The scoring models draw on transaction amount, velocity (frequency of transactions in a time window), geography (location deviation from customer history), and device fingerprint (new or unrecognized device), among other behavioral signals.

4.3 Explainability

Every score is decomposed using SHAP (SHapley Additive exPlanations) contribution values, so each decision lists the specific features that drove it (for example: transaction amount, new device, unusual location, velocity, and account history) rather than a single unexplained number.

4.4 Tech Stack

- Backend services and API gateway with JWT-based authentication and role-based access control (RBAC).
- Supervised learning: XGBoost for transaction-level fraud classification.
- Unsupervised learning: Isolation Forest for behavioral anomaly detection.
- Explainability library (SHAP) for per-decision feature attribution.
- Relational/document store for transactions, scores, and the analyst decision-feedback log.

5 Evaluation Criteria

Success is defined using measurable KPIs tied directly to the detection pipeline rather than subjective review:

- **Fraud Recall:** proportion of actual fraud correctly flagged by the system.
- **False Positive Rate:** proportion of legitimate transactions incorrectly flagged.
- **Mean Time to Detect:** latency between transaction occurrence and risk score generation.
- **Analyst Productivity:** change in average case-investigation time using the automated dashboard versus manual review.
- **Fraud Loss Avoided:** estimated monetary value of fraud prevented during the evaluation window.

Worked example: for a sample transaction of Rs. 85,000 against a customer’s normal range of Rs. 5,000–Rs. 10,000, the pipeline produces a fraud probability of 91% and a risk score of 87/100, driven primarily by transaction amount, an unrecognized device, and location deviation — triggering a Hold + Alert decision with a full attribution breakdown attached.

6 Scalability

The platform is designed to scale in both a theoretical and a practical sense.

- **Theoretical scalability:** the API gateway, transaction service, and detection stage are decoupled so each can scale independently under rising transaction volume.
- **Practical deployability:** stateless service design, indexed queries for the analyst dashboard, and a feedback database that supports incremental retraining without pipeline downtime.

7 Resources / Tooling

The project uses mature, freely available components so that effort concentrates on pipeline design, model quality, and explainability rather than infrastructure:

- Backend framework for REST APIs and the transaction/scoring services.
- XGBoost and scikit-learn (Isolation Forest) for the detection models.
- SHAP for explainability.
- A relational or document database for transactions, scores, and analyst feedback.
- No external funding required; all resources are available through open-source libraries and existing university/cloud compute access.

8 Project Scope and Deliverables

8.1 Initial Deliverable

- Transaction ingestion API with JWT + RBAC gateway.
- Rule engine and baseline XGBoost fraud model integrated into a single scoring call.
- Risk aggregation producing a 0–100 final score.
- Basic analyst dashboard displaying flagged transactions.

8.2 Subsequent Deliverables

- Isolation Forest anomaly detector added to the parallel detection stage.
- SHAP-based explainability attached to every score.
- Decision feedback database and analyst-confirmed relabeling loop.
- KPI reporting (recall, false-positive rate, mean time to detect, analyst productivity, fraud loss avoided).

9 Risks and Mitigations

- **Class imbalance (fraud is rare):** mitigate with resampling/weighted loss during XGBoost training and validate primarily on recall and precision rather than raw accuracy.
- **False positives blocking legitimate customers:** tune the risk-aggregation thresholds against the false-positive-rate KPI and keep a human-in-the-loop analyst review step.
- **Explainability overhead:** cache SHAP computations where possible to keep scoring latency within real-time bounds.
- **Data privacy:** minimize retained personally identifiable information and restrict dashboard access via RBAC.

10 Timeline

Phase	Focus
Weeks 1–2	Requirements, API gateway (JWT + RBAC), transaction service scaffolding.
Weeks 3–4	Rule engine and baseline XGBoost fraud model; risk aggregation and 0–100 scoring.
Weeks 5–7	Isolation Forest anomaly detector; analyst dashboard; parallel detection integration.
Weeks 8–9	SHAP explainability layer; decision feedback database and retraining loop.
Weeks 10–11	KPI instrumentation, evaluation against fraud recall / false-positive / MTTD targets.
Week 12	Buffer week for fixes; final testing and report/demo preparation.

11 Summary

FinGuard AI turns fraud detection from a static, rule-bound process into an adaptive, explainable risk platform. It layers deterministic rules with supervised and unsupervised machine learning, attaches a SHAP-based explanation to every decision, and closes the loop through analyst feedback — meeting the project’s engineering objectives through measurable KPIs (fraud recall, false-positive rate, mean time to detect, analyst productivity, and fraud loss avoided) rather than subjective assessment.